PRACTICAL EXAM

Subject Code	SWE202c
Subject Name	Introduction to Software Engineering
Page No	1
Topic	**Course Registration System**
Time	120 minutes
Tools	MS Word, Draw.io, MS Visio, Astah, Visual Paradigm, Star UML
Note	Open book. Students are NOT allowed to use the Internet.

I. CASE STUDY

Green University is a mid-sized institution with approximately 8,000 students and 400 academic staff. Each semester, the university must manage course enrollment for all undergraduate and postgraduate programmers. Currently, students submit paper-based registration forms to their faculty office, and clerks manually enter the data into spreadsheets. This process is error-prone, slow, and creates long queues during the registration window. The university's administration has therefore decided to commission a web-based Course Registration System (CRS) to automate and streamline the entire workflow.

The CRS is expected to serve four main groups of users. Students need to browse the catalogue of available courses, check their eligibility based on completed prerequisites, submit registration requests, and track the status of each request in real time. If a course is fully booked, a student may join an electronic waitlist; should a registered student later drop the course, the system automatically notifies and enrolls the first eligible student on the waitlist. Lecturers need to review the roster of students enrolled in their courses, record midterm and final grades, and release grades to students once the grading period closes. Academic Staff (administrators) are responsible for defining the semester timetable, setting and publishing course quotas, opening and closing the registration window, approving exceptional cases, and generating enrolment and grade-distribution reports. Finally, a System Administrator manages user accounts, configures system parameters, and oversees scheduled data backups.

From a technical standpoint, the university imposes several constraints. The system must remain responsive—handling at least 1,000 simultaneous users during peak registration periods with a page-response time under 3 seconds. Because student records and grade data are sensitive, all communication must be encrypted using TLS 1.2 or higher, and access must be governed by a role-based access control (RBAC) model so that each user sees only the functions and data relevant to their role. The platform must be available 99.5 % of the time during the semester, with any planned maintenance scheduled outside the registration window. The interface must be accessible on both desktop and mobile browsers and must conform to WCAG 2.1 Level AA accessibility standards. The architecture should also support horizontal scaling so that additional server capacity can be added smoothly as student numbers grow in future years.

II. QUESTIONS

Answer ALL questions. Base your answers on the Course Registration System case study above. Where diagrams are required, use standard UML notation and label all elements clearly.

Question 1: Software Development Model (2.0 points)

1. Identify and justify the most appropriate software development model (SDLC) for the CRS project. In your justification, address each of the following points: (1.0 points)
   - Whether requirements are fully known and stable at the outset
   - The need for iterative delivery and regular stakeholder feedback
   - Risk factors specific to the CRS (e.g., concurrent-user load, data security)
   - Team size, project duration, and the university's change-control process

2. Select ONE alternative SDLC model and compare it with your chosen model in the context of the CRS. Your comparison must include at least two advantages and two disadvantages for each model. (1.0 points)

Question 2: UC Modeling (1.5 points)

1. Identify ALL actors in the Course Registration System. For each actor, state their primary goal and whether they are a primary or secondary actor. (0.3 points)

2. List at least EIGHT use cases for the CRS. For each use case, state the initiating actor(s) and a one-sentence description of the goal. (0.4 points)

3. Draw a complete UML Use Case Diagram for the CRS. The diagram must include: (0.8 points)
   - All identified actors, placed correctly inside or outside the system boundary
   - All major use cases inside the system boundary rectangle
   - At least ONE <<include>> relationship — provide a brief written rationale
   - At least ONE <<extend>> relationship — provide a brief written rationale
   - Any actor-generalisation relationships that apply

Question 3: UC Specification (1.5 points)

Write a complete Use Case Specification for the use case 'Register for Course'. Use the template in the answer sheet and fill in EVERY field. Partial or missing sections will receive partial credit only.

Question 4: Non-Functional Requirements (1.0 points)

1. From the case study, identify and classify at least FOUR non-functional requirements (NFRs) into appropriate quality categories (e.g., Performance, Security, Availability, Usability, Scalability, Maintainability). For each NFR, write one measurable acceptance criterion. Present your answer as a table with the columns: Category | NFR Description | Acceptance Criterion. (0.9 points)

2. Identify TWO NFRs from your table that may conflict with each other in the CRS context. Clearly explain why the conflict arises and propose a concrete trade-off strategy that the development team could apply. (0.6 points)

Question 5: Class Diagram (1.5 points)

1. Identify at least SIX key domain classes for the Course Registration System. For each class, specify at minimum three attributes (with data types) and two methods. Present this as a structured list or mini class-description table before drawing the diagram. (0.6 points)

2. Draw a complete UML Class Diagram that includes: (1.4 points)

- All identified classes with their full attribute and method compartments
- At least ONE inheritance (generalization) relationship — for example, User as a base class for Student, Lecturer, and AcademicStaff
- At least TWO association relationships with correct multiplicity labels at both ends
- At least ONE aggregation OR composition relationship, with the correct diamond notation and a written justification
- At least ONE dependency relationship

Suggested classes (you may add or rename): User, Student, Lecturer, AcademicStaff, SystemAdmin, Course, CourseOffering, Registration, Waitlist, Grade, Semester, Report.

Question 6: AI in coding (1.0 points)

This question assesses your ability to use AI coding assistants (e.g., GitHub Copilot, ChatGPT, Claude) as a practical aid during software development. Two Java code snippets from the Course Registration System are provided below. For each scenario, you must craft an effective AI prompt and then answer the follow-up tasks

based on what the Al would identify.

Important: A good AI prompt must include: (1) clear context about the system, (2) the specific goal (review / find bug / fix), (3) the code itself, and (4) the expected output format.

Scenario A- RegistrationService.java (0.5 points)

The following Java method is part of the RegistrationService class in a Spring Boot application. It is intended to enroll a student in a course by (1) verifying the prerequisite is satisfied, (2) confirming a seat is available, and (3) persisting the new registration record. The code compiles without errors but contains THREE logical or semantic bugs.

```
public class RegistrationService {
    public boolean registerStudent(Student student, Course course) {
        // Check prerequisite
        if (course.getPrerequisite() != null &&
            !student.getCompletedCourses().contains(course.getPrerequisite())) {
            return true;  // Bug 1
        }
        // Check available seats
        if (course.getEnrolled() >= course.getMaxSeats()) {  // Bug 2
            return false;
        }
        // Enroll student and persist
        course.getRoster().add(student);
        registrationRepo.save(new Registration(student, course));  // Bug 3
        return true;
    }}
```

Your tasks for Scenario A:

- **Write an AI Prompt for Code Review**. Write a clear and complete prompt you would submit to an AI assistant to review the registerStudent() method. Your prompt must specify: the system context (CRS, Java, Spring Boot), what you want the AI to do (review for correctness, logic errors, and coding best practices), and the expected output format (e.g., a numbered list of issues with explanations and suggested fixes). (0.2 points)
- **Identify the Bugs**. Based on the AI review, identify all THREE bugs. For each bug state: the line or code fragment, the bug type (e.g., wrong return value, off-by-one, missing state update), and why it causes incorrect behaviour. (0.2 points)
- **Write an AI Prompt to Fix the Bugs.** Write a follow-up prompt asking the AI to provide the corrected Java method. Present the corrected code with an inline comment on each fix. (0.1 point)

Scenario B - GradeService.java (0.5 points)

The following Java method belongs to the GradeService class. It should publish final grades only when (1) the grading period is officially closed and (2) every enrolled student has a grade on record. The code contains TWO logical bugs and ONE critical security vulnerability.

```
public class GradeService {
    private EntityManager em;
    public String publishGrades(String courseId) {
        // Fetch course from database
        Course course = (Course) em.createNativeQuery(
            "SELECT * FROM courses WHERE id = '" + courseId + "'"  // Security bug
        ).getSingleResult();
        // Guard: grading period must be closed first
        if (!course.isGradingPeriodClosed()) {  // Logical bug 1
            return "Grades published";
        }
        // Guard: all students must have a grade
        if (course.getGrades().size() == course.getRoster().size()) {  // Logical bug 2
            return "Not all grades entered";
        }
        course.setGradesPublished(true);
    return "Grades published"; }}
```

Your tasks for Scenario B:

- **Write an AI Prompt for Security and Logic Review.** Write a prompt asking an AI assistant to review publishGrades() for both logical correctness and security vulnerabilities. Specify the language and framework (Java / JPA), describe the method's intended behaviour, and request that each issue be explained with a risk level (Low / Medium / High / Critical). (0.2 points)
- **Identify All Issues**. Based on the AI review, explain all three issues: the two logical bugs (which condition is wrong and what the correct logic should be) and the security vulnerability (name the vulnerability type, describe how an attacker could exploit it in the CRS, and assign a risk level). (0.2 points)
- **Write an AI Prompt to Fix the Issues**. Write a follow-up prompt instructing the AI to provide the fully corrected Java method, replacing the native query with a JPA named query (parameterised). Present the corrected code with inline comments for each fix. (0.1 point)

Question 7: Testing Stage and Type (1.5 points)

1. Map the four standard testing stages to the CRS development process. For each stage, state what is tested, who is responsible, and which testing types are most applicable: (0.5 points)
2. Write THREE test cases for the 'Register for Course' use case, covering the scenarios listed below. Present each test case in a table with the columns: Test Case ID | Test Case Name | Precondition | Test Steps | Expected Result | Testing Type. (1.0 points)
   - Happy-path scenario - a student successfully registers for an available course
   - Boundary / edge-case scenario - a student registers for the last available seat (quota = 1 remaining)
   - Negative / error scenario - a student attempts to register without meeting the course prerequisite

III. SCORING RUBRIC

Q#	Topic	Max Score	Weight	Grader's Score
1	Software Development Model Selection	2.0	20%	
2	UC Modeling	1.5	15%	
3	UC Specification	1.5	15%	
4	Non-Functional Requirements	1.0	10%	
5	Class Diagram	1.5	15%	
6	AI in Coding	1.0	10%	
7	Testing Stage and Type	1.5	15%	
TOTAL		**10**	**100%**	

IV. SUBMISSION GUIDELINES

4.1. Submission Format

- All work must be submitted as a single .zip file.
- **File naming**: StudentID _SWE202c.zip (e.g., SE123456_SWE202c.zip)
- The .zip must contain: Answer document in MS Word .docx

4.2. Formatting Requirements

- Insert the answers below the question. Each question should begin on a new page where possible.
- Clearly label the question number and sub-question before each answer section.
- UML diagrams must be clear, titled, and use correct standard notation.
- Font: Times New Roman or Arial, size 12–13pt, line spacing 1.5.

4.3. General Evaluation Criteria

- Content accuracy and completeness as required by each question.
- Correct UML syntax, clear and well-annotated diagrams.
- Logical reasoning tied to the specific context of the CRS system.
- Clean, professional presentation.

— END OF EXAM —

Label all diagrams clearly and justify your design decisions wherever asked.